



박솔민

👤 사람	siribok
🕒 최종 편집 일시	2026년 9월 13일 오후 3:28

▼ 중간 발표 대본

안녕하세요! 에레렘팀 박솔민입니다.

오늘 AI한테 뭐 하나라도 물어보고 오신 분?

다 하셨죠. 그럼 하나만 여쭙볼게요.

그 대화, 삭제하면 진짜 사라질까요?

저희가 5주 동안 분석한 내용이 이 질문입니다. 그럼 시작하겠습니다.

순서는 네 개인데, 오늘 진짜 발표 내용은 3장 분석 결과입니다. 앞은 빨리 진행하겠습니다.

저희는 김준범 멘토님, 김종현 PL님, 그리고 팀원 여덟 명입니다. 여덟 명이 서비스 하나씩 붙잡고 5주를 걸었습니다.

주제는 이겁니다. LLM 대화 데이터를 분석해서 사용자 행위를 재구성하고, 포렌식 파서와 가이드라인 까지 만드는 것입니다. 대상은 ChatGPT, Claude, Gemini, LM Studio 네 가지이고

또 저희는 욕심을 내서 같은 서비스도 웹이랑 앱이 다르거든요. 그래서 여덟 트랙으로 쪼갰습니다.

목표는 다섯 단계입니다. 저장 구조 분석, 행위별 흔적 수집까지 끝냈고, 지금 3단계 타임라인 재구성 중입니다. 남은 건 가이드라인과 파서개발입니다.

그럼 이 프로젝트를 왜 하느냐. 국내 생성형 AI 경험률 39%입니다. 이미 일상이 되었죠. 또 대화를 조작해서 허위 증거로 내는 사례가 실제로 나오고 있습니다. LLM 대화가 새로운 디지털 증거가 된 겁니다.

그럼 기존 포렌식 도구는 있을까요? 저희가 찾아본 바로는 없습니다. 대화를 조작해서 허위 증거로 내는 사례가 실제로 나오고 있습니다. LLM 대화가 새로운 디지털 증거가 된 겁니다. 증거는 늘어나는데 도구가 없다. 그래서 저희가 뛰어들었습니다.

WBS입니다. 체크된 데까지 완료되었고, 지금 딱 여기가 중간발표고요. 9월엔 파서개발, 10월엔 학술대회 논문 초안 작성할 계획입니다.

1주차엔 선행 연구를 봤고, 2주차엔 네 서비스의 웹·앱 아티팩트를 전수 조사했습니다.

3~4주차는 사용자 행위를 33개로 정의했어. 그리고 여덟 명이 똑같은 환경에서 33개를 다 해봤습니다. 행위 전 스냅샷, 행위, 행위 후 스냅샷. 파일 단위로 전부 비교했습니다.

5주차에 한 표로 합쳤는데, 똑같은 행위여도 서비스마다 흔적 남는 곳이 다 달랐어요.

저희 환경은 Golden 이미지 Windows 11로 전원 동일 환경, 웹은 Chrome만. 도구는 FTK Imager, NTFS Log Tracker, Procmon, WinHex를 썼고요. 오른쪽이 33개 행위를 10개로 묶은 표입니다.

절차는 다섯 단계입니다. 행위 전과 수행 후 2단계로 템플릿 설계를 했고 행위 재현 후 파일시스템, 레지스트리, 프로세스 아티팩트를 직접 관찰했습니다. 그리고 저장 위치와 형식, 변화 내용 기록을 하였고 저희가 설정한 도구들로 교차 검증을 했고 증거 수준을 태깅까지 했습니다.

결과입니다. 서비스마다 확인한 게 훨씬 많은데, 오늘은 대표 두 개씩만 뽑았습니다.

Claude 웹. 첫째, 대화를 삭제했더니 IndexedDB에선 사라졌는데 Cache에 원문이 그대로 있었습니다. 내용, UUID, 시간, 모델명까지요.

화면에서 지운 거지, 디스크에서 지운 게 아니었다는 것을 알 수 있었습니다. 둘째, 다운로드 파일을 휴지통으로 지웠는데 \$MFT에 다운로드 경로가 남았습니다. 한계로는 휴지통까지만 확인했고 영구 삭제는 미검증입니다.

Claude 데스크톱입니다. 로그아웃하면 뭐가 지워질까요? 저는 쿠키 정도겠지 했는데 아니었습니다. IndexedDB 파일 여섯 종이 동시에 삭제되고 0.05초 만에 전부 재생성됩니다. 감소율 99.6%, 1차실험과 2차실험에서 재현까지 됐습니다. 근데 같은 시점에 HTTP Cache의 대화 본문은 멀쩡했습니다. 네 저장소 중 한 칸만 없앤거예요. Cache까지 보면 전문이 복원됩니다. 둘째, 계정 이름 변경값이 평문으로 남는데 이메일·계정 UUID·조직 UUID가 한 레코드에 통째로 들어 있습니다.

ChatGPT 웹입니다.

첫째, 로그아웃은 URL, Local Storage, 쿠키 세 개를 교차 확인해야 판별됩니다. 쿠키만 보면 세션 만료량 구분이 안 돼요.

둘째, 캐시를 삭제해도 NTFS에 흔적이 남습니다. \$UsnJrnl에 FileDelete 14건이 그대로 잡혔습니다. 데이터는 없어져도, 지웠다는 행위는 남는다는 것을 알 수 있습니다.

ChatGPT 데스크톱입니다. 첫째, 앱을 지우고 대화도 지웠는데 auth.json이 남았습니다. 대화, JWT, 계정 식별자까지 복구됐어요. 저널을 보니 .codex 폴더는 DELETE 기록이 아예 없었습니다. 앱 삭제가 그 경로를 안 건드린 다는걸 알 수 있습니다.

둘째, 프롬프트가 평문으로 남습니다. 수정하면 원본을 안 지우고 분기 파일을 새로 만들어서 수정 전후가 둘 다 남습니다.

그다음 Gemini입니다. 첫째, 문서 다운로드 History에 파일명·경로·시각까지 남고 실제 파일도 확인했습니다. 그다음 이미지 생성을 요청했는데 브라우저 캐시에서 생성 이미지 원본을 그대로 복구했습니다. 그게 저 그림입니다. 게다가 복구한 이미지, 대화 제목, 이미지 쿠키 독립 채널 세 개가 같은 시점 같은 대상을 가리켰습니다. 하지만 캐시 특성상 수집이 늦으면 캐시 복구는 실패할 수 있습니다.

마지막 LM Studio, 로컬 LLM입니다.

첫번째는 앱을 지우면 실행파일은 사라지는데 대화 기록, 모델, 인증 정보, 서버 로그는 그대로 남습니다. 앱 삭제만으로는 아무것도 못 지웁니다.

둘째로는 문서를 올리면 원문 텍스트가 파일로 저장되는데 크기에 따라 JSON이나 RAG 데이터베이스로 갈립니다. 어느 쪽이든 원문이 남습니다.

최종 산출물에 대해 말씀드리겠습니다.

첫 산출물은 LLM 포렌식 분석 가이드라인입니다. 여덟 트랙에서 나온 공통점과 차이점을 어떤 순서로 뭉쳐 봐야 하는지로 바꾸는 겁니다.

두 번째는 도구입니다. 네 가지를 만듭니다.

첫번째는 아티팩트 자동 수집·파싱.

그다음은 행위-아티팩트 상관관계 분석입니다. 이 흔적이 어떤 행위였는지 역추적 하는 것이지요.

세번째는 다중 아티팩트 Evidence Correlation. 입니다. 아까 Gemini에서 채널 세 개가 한 점을 가리킨 것처럼, 엮어서 확신도를 올립니다.

그리고 수사 지원 Intelligence 제공까지 계획했습니다.

또 파일을 덤프하는 도구가 아니라, "이 사람이 언제 뭘 했다"를 말해주는 도구가 목표입니다.

이 도구가 어디 쓰이나. 분석 대상은 저희가 직접 확인한 저장소들까지이고
 행위와 흔적은 33개 매핑 테이블로 연결하고, 결과는 타임라인으로 냅니다.
 그래서 허위 증거 사건에서, 화면에 보이는 대화가 아니라 디스크에 남은 원문으로 진위를 가릴 수 있게
 됩니다.

네 이상으로 에레렘팀 발표를 마치겠습니다. 감사합니다.

▼ 요구사항 명세서

AI 포렌식 파서 요구사항 명세서

PART 1. 개요 및 제품 범위

1.1 목적

본 도구는 조사관·보안 분석가가 **오프라인 디스크 이미지만으로** LLM 사용 이력을 재구성할 수 있도록
 지원한다. 본 명세서는 설계와 시험의 기준 문서로 사용한다.

1.2 분석 대상 클라이언트

대상	클라이언트 형태	핵심 저장 기술
● Claude Web	Chrome 프로파일	LevelDB · IndexedDB · HTTP 캐시 (zstd)
● Claude Desktop	MSIX + Electron(Chromium)	LevelDB · blob · regf 하이버
● ChatGPT Web	Chrome 프로파일	LevelDB(conversationDrafts) · SQLite
● ChatGPT Desktop (Codex)	MSIX + 내장 Chromium	JSONL · SQLite · JWT
● Gemini Web	Chrome 프로파일	History URL · Cache_Data · ADS
● LM Studio	로컬 LLM 데스크톱 앱	JSON · SQLite(RAG) · 서버 로그

1.3 제품 범위

파서가 수행하는 것	❌ 파서가 수행하지 않는 것
오프라인 이미지·볼륨·파일 트리에서 아티팩트 식별·파싱·복 호	실시간 모니터링
아티팩트를 사용자 행위로 매핑	서비스 제공자 API 호출
다중 아티팩트 상관 분석	네트워크 패킷 수집
통합 타임라인 및 근거 인용 산출	대화 내용의 의미 분석·요약
삭제 데이터 복구	사용자 의도의 법적 판단

1.4 핵심 용어

용어	정의
아티팩트	사용자 행위의 결과로 저장매체에 남는 데이터. 파일·레코드·메타데이터 포함
직접 증거	해당 행위 고유의 아티팩트로서 단독으로 행위를 지시하는 근거
보조 증거	다른 행위에서도 발생할 수 있어 단독으로는 행위를 특정하지 못하는 근거
보완적 소거	소거 행위마다 비워지는 저장소가 달라, 단일 소거로는 데이터가 완전히 지워지지 않는 구조
ADS	Alternate Data Stream. NTFS 부수 스트림. Zone.Identifier 가 다운로드 출처 표시
MSIX	Windows 앱 패키지 형식. 파일·레지스트리 접근이 가상화됨
MFT	NTFS Master File Table. 파일 메타데이터와 4중 시각 보관
LevelDB	Chromium의 Local Storage-IndexedDB가 사용하는 키-값 저장소

PART 2. 시스템 구조 및 운영 환경

2.1 처리 파이프라인 (6단계)

#	구성요소	역할
1	수집기 Collector	이미지·볼륨·파일 트리에서 아티팩트를 읽기 전용 획득
2	클라이언트 파서 Parser	대상별 저장 구조 해석 → 원시 레코드 추출
3	디코더 Decoder	LevelDB·SQLite·JSONL·zstd·regf 형식별 복호
4	복구기 Recovery	삭제 엔트리·비할당 영역에서 소실 데이터 복원
5	정규화·매핑 Normalizer	통합 스키마 변환 후 사용자 행위로 매핑
6	상관·출력 Correlator / Reporter	다중 채널 교차 검증 → 타임라인·보고서 생성

[이미지 / 볼륨 / 파일트리]

```

  |
  ① 수집 → ② 파싱 → ③ 디코딩 → ④ 복구
  |
  ⑤ 정규화 · 사용자 행위 매핑
  |
  ⑥ 상관 분석 → 통합 타임라인 · 보고서 (JSON / XLSX / HTML)

```

2.2 제품 핵심 기능

- 6개 대상 클라이언트의 아티팩트 **자동 탐지 및 파싱**
- Chromium 계열 저장소(LevelDB·HTTP 캐시·SQLite) 및 앱 고유 형식(JSONL·regf) **복호**
- zstd·gzip 압축 응답 본문 해제 및 **UTF-8 / UTF-16LE 동시 검색**
- 삭제된 MFT 엔트리·비할당 영역으로부터의 **데이터 복구**
- 사용자 행위 판별 및 **증거 수준 태깅**
- 다중 아티팩트 상관 분석을 통한 **확신도 산출**

- 통합 타임라인 생성 및 근거 인용 포함 보고서 출력

2.3 사용자 특성

사용자	전문성	주요 사용 목적
 디지털포렌식 조사관	NTFS·브라우저 아티팩트 이해, 도구 사용 숙련	사건 대상 PC의 LLM 사용 이력 재구성
 보안 분석가	로그·아티팩트 분석 경험	내부 정보 유출 경로로서 LLM 사용 여부 확인
 연구자	저장 구조·파싱 원리 이해	서비스별 아티팩트 비교 연구 및 규칙 확장

2.4 운영 환경

구분	내용
분석 수행 환경	Windows 10/11 x64, Python 3.11+, 메모리 8GB 이상, SSD 권장
분석 대상 환경	Windows 10/11 NTFS 볼륨
입력 형식	디스크 이미지(RAW · E01 · VMDK), 마운트된 볼륨, 추출된 파일 트리
출력 형식	JSON(기계 판독), CSV · XLSX(표), HTML · PDF(보고서)
네트워크	불필요 — 완전 오프라인 동작

2.5 설계 및 구현 제약

제약 사항
분석 대상 이미지는 읽기 전용으로만 접근한다
관리자 권한 없이 동작한다. 볼륨 마운트를 전제하지 않고 이미지 직접 파싱을 기본 경로로 한다
상용 GUI 포렌식 도구(FTK Imager·EnCase 등)에 의존하지 않는다. 교차 검증용으로만 사용한다
외부 네트워크 접속 없이 동작한다
서비스 제공자 API를 호출하지 않는다. 로컬 아티팩트만을 근거로 한다

2.6 가정 및 의존성

내용
분석 대상은 Windows 10/11 NTFS 볼륨이다
클라이언트 버전이 인벤토리 기준과 다를 경우 경로 규칙 갱신이 필요하다
웹 대상은 Chrome 브라우저를 전제한다. 타 브라우저는 향후 확장 대상이다
대상별 경로·판별 규칙은 아티팩트 인벤토리에 의존한다. 인벤토리 갱신 시 규칙 파일도 갱신한다
zstd · LevelDB · SQLite 파싱은 외부 오픈소스 라이브러리에 의존한다

PART 3. 기능 요구사항

| 우선순위 ● 필수 · ● 권장 · ● 선택

3.1 외부 인터페이스

기본 인터페이스는 **CLI**이며, 단일 명령으로 수집부터 보고서 생성까지 실행한다.

```
aifp analyze --input D:\case\image.vmdk --targets all --out D:\case\result --format json,xlsx,html
```

대상	인터페이스	용도
디스크 이미지 리더	라이브러리 호출	VMDK 체인 · RAW 파티션 읽기
NTFS 파서	라이브러리 호출	\$MFT · \$UsnJrnl · \$LogFile 해석
LevelDB 리더	라이브러리 호출	Local Storage · IndexedDB 레코드 열거
SQLite 엔진	표준 라이브러리	History · Cookies · 상태 DB 조회
압축 해제기	라이브러리 호출	zstd · gzip · brotli 응답 본문 해제
규칙 파일	파일 입출력	행위 판별 규칙(YAML / JSON) 로딩

- **하드웨어 인터페이스** — 물리 디스크 직접 접근 불요. 파일 단위 읽기만으로 동작한다
- **통신 인터페이스** — 해당 없음. 네트워크 통신을 수행하지 않는다

3.2 수집 및 입력

	명칭	내용	우선
01	입력 소스 수용	디스크 이미지(E01/RAW·VMDK), 마운트된 볼륨, 추출된 파일 트리 3종을 입력으로 받는다	●
02	원본 읽기 전용 보장	입력 소스에 쓰기 연산을 수행하지 않으며, 처리 전후 소스 해시 동일성을 로그로 남긴다	●
03	대상 자동 탐지	분석 대상 클라이언트의 루트 경로를 자동 탐색하고 발견 / 미발견 대상을 구분해 보고한다	●
04	사용자 프로파일 다중 처리	동일 이미지 내 복수 Windows 계정 및 복수 Chrome 프로파일을 각각 독립 처리한다	●
05	수집 시점 기록	각 아티팩트의 획득 시각을 UTC로 기록하고 원본의 NTFS 4시각을 함께 보존한다	●

3.3 대상별 파싱

	명칭	내용	우선
01	Claude Web	History·Cookies·Local Storage·IndexedDB·Cache_Data·Session/Tabs를 파싱해 대화 UUID, 제목, 메시지 본문, 첨부 메타 데이터, 계정·조직 UUID 를 추출한다	●
02	Claude Desktop	config.json·IndexedDB(leveldb / blob)·Local Storage·Cache_Data·Helium 하이브·Prefetch를 파싱한다. MSIX 가상화 경로(LocalCache\Roaming) 를 기본 루트로 사용한다	●
03	ChatGPT Web	History·Cookies·Local Storage(conversationDrafts)·Cache_Data를 파싱해 대화 URL, 공유 링크, 초안, 선택 모델값 을 추출한다	●
04	ChatGPT Desktop	auth.json·codex-global-state.json·rollout-*.json·session_index.json·상태 SQLite를 파싱해 대화 전문, 프롬프트 이력, JWT, 계정 식별자 를 추출한다	●
05	Gemini Web	History(대화·로그인·로그아웃·프로필 변경 URL)·Cookies·Cache_Data를 파싱하고 대화 ID를 URL에서 추출 한다	●
06	LM Studio	conversation JSON·retrieval-database.db·user-files 메타데이터·server-logs·계정 프로필 JSON을 파싱한다	●
07	파서 플러그인 구조	클라이언트 파서는 공통 인터페이스를 구현하는 플러그인으로 분리하며, 신규 서비스 추가 시 코어 수정 없이 파서 모듈만 추가 한다	●
08	버전 지문 기록	파싱한 클라이언트의 버전을 식별해 결과에 기록한다	●

3.4 저장 형식 디코딩

	명칭	내용	우선
01	LevelDB 디코딩	Local Storage·IndexedDB의 LevelDB(.log · .ldb ·MANIFEST)를 읽고 삭제 표시 레코드를 포함 해 열거한다	●
02	IndexedDB 값 역직렬화	V8 직렬화 형식을 해석하고, 1바이트 / 2바이트 문자열을 구분해 UTF-16LE 문자열을 정상 복원 한다	●
03	HTTP 캐시 파싱	Chromium 디스크 캐시(blockfile: index · data_0~3 · f_*)를 파싱해 요청 키와 응답 본문을 분리한다	●
04	응답 본문 압축 해제	zstd·gzip·brotli 본문을 자동 판별해 해제한다. 판별은 헤더가 아니라 매직바이트(zstd 28 B5 2F FD) 를 우선한다	●

	명칭	내용	우선
05	SQLite 파싱	History·Cookies·상태 DB를 <code>-wal</code> · <code>-shm</code> 과 함께 읽어 미체크포인트 레코드를 포함한다	●
06	JSONL 스트림 파싱	줄 단위 JSON을 파싱하며, 손상된 줄은 건너뛰고 위치를 기록한다	●
07	레지스트리 하이브 파싱	MSIX 하이브 <code>SystemAppData\Helium\User.dat</code> 과 트랜잭션 로그 LOG1·LOG2를 함께 읽어 미반영 쓰기를 복원한다	●
08	이중 인코딩 검색	모든 문자열 검색은 UTF-8과 UTF-16LE를 동시에 수행한다. 단일 인코딩 검색은 허용하지 않는다	●

3.5 삭제 아티팩트 복구

	명칭	내용	우선
01	MFT 삭제 엔트리 열거	<code>\$MFT</code> 에서 <code>in_use=0</code> 엔트리를 열거해 파일명·크기·4시각·상주 속성을 복원한다	●
02	ADS 복원	<code>Zone.Identifier</code> 대체 데이터 스트림을 상주 속성에서 복원해 다운로드 출처(Zoneld)를 제시한다	●
03	비할당 영역 카빙	zstd·JPEG·PNG 시그니처 기반으로 비할당·삭제표시 클러스터를 카빙하고 해제 후 텍스트 검색한다	●
04	복구 신뢰도 표기	복구 결과마다 완전 복구 / 부분 복구 / 메타데이터만 복구를 구분해 표기한다	●
05	<code>\$UsnJrnl</code> 파싱	<code>\$UsnJrnl:\$J</code> 를 파싱해 파일 생성·삭제·이름 변경·ADS 기록을 시간순으로 제공하며, 부모 경로는 통합 참조 맵으로 복원한다	●

3.6 정규화

	명칭	내용	우선
01	통합 이벤트 스키마	대상별 파싱 결과를 PART 4의 단일 이벤트 스키마로 정규화한다	●
02	시각 정규화	모든 시각을 UTC로 저장하고 표시 시 타임존을 지정한다. FILETIME · Unix epoch · WebKit epoch를 각각 변환한다	●
03	식별자 정규화	계정 UUID·조직 UUID·대화 UUID·파일 UUID를 유형별로 분류해 저장한다	●
04	중복 제거	동일 아티팩트가 복수 경로에서 검출되면 하나의 이벤트로 병합하고 출처 목록을 보존한다	●

3.7 사용자 행위 매핑

	명칭	내용	우선
01	행위 매핑 규칙	PART 6의 사용자 행위에 대해 대상별 판별 규칙을 규칙 파일로 정의하고, 파서는 이 규칙을 해석해 행위를 판정한다	●
02	증거 수준 태깅	각 판정에 직접 증거 / 보조 증거 / 확인 어려움 3등급을 부여한다	●
03	미적용 구분	대상 클라이언트에 해당 기능이 없어 판정 불가능한 행위는 미검출이 아니라 “해당 없음” 으로 구분 출력한다	●
04	규칙 외부화	판별 규칙은 코드가 아닌 외부 규칙 파일(YAML / JSON)로 관리해 서비스 변경 시 코드 수정 없이 갱신한다	●
05	부정 결과 보고	규칙이 요구하는 아티팩트가 없어 판정하지 못한 경우, 그 사실과 확인한 경로를 함께 보고한다	●

3.8 다중 아티팩트 상관 분석

	명칭	내용	우선
01	교차 검증	동일 행위를 지시하는 서로 독립적인 아티팩트 채널 수를 세어 확신도를 산출한다	●
02	시간 근접 상관	설정 가능한 시간 창(기본 ±5초) 내 아티팩트를 하나의 행위로 묶는다	●
03	저장소 분할 대조	한 저장소가 비어 있어도 다른 저장소의 잔존 여부를 함께 조사해 보완적 소거 구조 를 반영한다	●
04	상충 결과 표기	대상 간·아티팩트 간 결과가 상충하면 임의로 하나를 선택하지 않고 상충 사실을 표기 한다	●
05	계정-대화 연결	계정 식별자와 대화 UUID를 연결해 한 PC의 복수 계정 사용 이력 을 열거한다	●

3.9 타임라인 생성

	명칭	내용	우선
01	통합 타임라인	모든 대상 클라이언트의 이벤트를 하나의 시간축에 병합 해 출력한다	●
02	시각 출처 표기	각 이벤트의 시각이 \$UsnJrn1 ·MFT·애플리케이션 로그·API 응답 중 어디서 왔는지 표기한다	●
03	시각 신뢰도 경고	MFT 시각이 앱 종료 시각으로 수렴한 것으로 판단되면 해당 시각에 신뢰도 경고 를 부여한다	●
04	필터링	기간·대상·행위 분류·증거 수준·확신도로 타임라인을 필터링한다	●

3.10 출력 및 보고

	명칭	내용	우선
01	기계 판독 출력	전체 결과를 JSON 으로 출력하며 스키마 버전을 포함한다	●
02	표 출력	행위별 요약 CSV·XLSX로 출력한다	●
03	보고서 출력	조사관용 요약 보고서를 HTML 또는 PDF로 출력한다	●
04	증거 인용	모든 결론에 원본 파일 경로·오프셋·해시 를 인용으로 첨부한다	●
05	원문 추출	복원한 프롬프트·응답 원문을 개별 파일로 내보낸다	●

3.11 무결성 및 감사

	명칭	내용	우선
01	해시 기록	입력 소스와 추출 산출물의 SHA-256 을 기록한다	●
02	처리 로그	수행한 모든 연산을 시각·대상·결과와 함께 감사 로그 로 남긴다	●
03	재현 가능 실행	동일 입력·동일 버전·동일 규칙 파일에서 동일 출력 을 산출한다	●
04	버전 고정	도구 버전과 규칙 파일 버전을 결과에 기록한다	●

PART 4. 비기능 요구사항 및 데이터 스키마

4.1 성능·비기능 요구사항

ID	속성	내용	우선
PER-01	성능	64GB NTFS 볼륨 1개의 MFT 전수 파싱 + 지정 경로 추출을 5분 이내 완료한다	●
PER-02	성능	단일 대상의 아티팩트 파싱을 60초 이내 완료한다 (SSD, 8코어 기준)	●
PER-03	성능	메모리 사용량은 4GB를 넘지 않으며 , 대용량 파일은 스트리밍 방식으로 처리한다	●
REL-01	신뢰성	파서 하나가 예외로 실패해도 나머지 대상 처리는 계속되며, 실패 사실을 결과에 기록한다	●
REL-02	신뢰성	손상·부분 수집 아티팩트를 만나도 중단하지 않고 부분 결과와 손상 위치 를 보고한다	●
SEC-01	보안성	추출한 토큰·쿠키·개인정보는 기본 출력에서 마스킹 하고, 명시적 옵션에서만 원문을 출력한다	●

ID	속성	내용	우선
SEC-02	보안성	도구는 네트워크 통신을 수행하지 않는다 (오프라인 동작)	●
USA-01	사용성	CLI를 기본 인터페이스로 제공하며 단일 명령으로 전 과정을 실행 할 수 있다	●
USA-02	사용성	처리 진행률과 남은 대상을 표시한다	●
MNT-01	유지보수성	파서 추가는 코어 코드 변경 없이 플러그인 등록만으로 가능하다	●
MNT-02	유지보수성	경로·규칙은 코드에서 분리해 외부 파일로 관리한다	●
POR-01	이식성	Windows 10/11 x64에서 동작하며, Python 3.11 이상 표준 배포판에서 실행된다	●
POR-02	이식성	분석 대상 OS와 수행 OS를 분리해, 오프라인 이미지 분석 시 대상 OS 부팅을 요구하지 않는다	●

4.2 소프트웨어 시스템 속성

속성	요구 내용
신뢰성	개별 파서 실패가 전체 처리를 중단시키지 않으며, 부분 수집·손상 아티팩트도 부분 결과를 산출한다
가용성	오프라인 단독 실행으로 외부 서비스 가용성에 의존하지 않는다
보안성	인증 토큰·쿠키·개인정보는 기본 마스킹한다. 산출물은 사건 디렉터리 밖으로 기록하지 않는다
유지보수성	파서는 플러그인으로, 판별 규칙은 외부 파일로 분리한다
이식성	Windows 10/11 x64 + Python 3.11 표준 배포판에서 동작한다
검증 가능성	동일 입력에서 동일 출력을 산출하며 감사 로그로 재현을 증명한다

PART 5. 대상별 아티팩트 인벤토리

| 파서 대상 경로의 원본

● Claude Web — Chrome 프로파일

저장소	아티팩트 / 추출 정보
History	urls · visits · visited_links · downloads · downloads_url_chains
Cookies	sessionKey · sessionKeyLC · lastActiveOrg · activitySessionId · anthropic-device-id
Local Storage (leveldb)	static-composer-draft-data · __qk_hint_account_uuid · analytics_queue
IndexedDB	react-query-cache (account_info · chat_conversation_list · chat_conversation_tree) · attachment-blob · chat-draft
Cache_Data	chat_conversations API 응답 (zstd 압축)

저장소	아티팩트 / 추출 정보
Session · Tabs	NavigationEntry
Login Data	자격 증명 저장 항목

Claude Desktop — MSIX + Electron(Chromium)

루트 %LOCALAPPDATA%\Packages\Claude_pzs8srxjxfjjc\LocalCache\Roaming\Claude\

저장소	아티팩트 / 추출 정보
설정	config.json → lastKnownAccountUuid · 조직 UUID · oauth:tokenCache
IndexedDB	https_claude.ai_0.indexeddb.leveldb\000003.log · ...indexeddb.blob\1\00*
Local Storage	leveldb\000005.ldb
캐시	Cache\Cache_Data (f_* · data_* , zstd) · Code Cache · GPUCache
네트워크·스토리지	Network\Cookies · WebStorage\CacheStorage · blob_storage
앱 상태 파일	lockfile · main.log · window-state.json · plan-usage-history.json · fcache
레지스트리	SystemAppData\Helium\User.dat (+ LOG1 · LOG2)
로그·실행 흔적	%LOCALAPPDATA%\Claude\logs · Prefetch\CLAUDE.EXE-*.pf
파일시스템	\$MFT · \$UsnJrnl:\$J · \$LogFile · Zone.Identifier ADS

ChatGPT Web — Chrome 프로파일

저장소	아티팩트 / 추출 정보
History	/auth/logout · chatgpt.com/s/<share> · conversation URL
Cookies	nextauth 계열 인증 쿠키 · nextauth.message=signout
Local Storage	leveldb\000007.log → oai/apps/conversationDrafts
Cache_Data	응답 본문
환경설정	Preferences · Secure Preferences · Session · Tabs
모델 선택값	model_slug · selected_model · default_model

ChatGPT Desktop (Codex) — MSIX + 내장 Chromium

저장소	아티팩트 / 추출 정보
인증	~\.codex\auth.json → JWT(id / access / refresh) + account_id
전역 상태	~\.codex\.codex-global-state.json → prompt-history · thread-descriptions · client-thread-bindings-v1 · chatgpt-last-selected-model-v1
세션 전문	~\.codex\sessions\<Y>\<M>\<D>\rollout-<ts>-<id>.jsonl
세션 색인	~\.codex\session_index.jsonl

저장소	아티팩트 / 추출 정보
락	~\.codex\thread-writer-locks\coordination.lock · <id>.lock
상태 SQLite	goals_1 · logs_2 · memories_1 · queue_1 · state_5 · thread_history_1 (+ -wal · -shm)
패키지 캐시	%LOCALAPPDATA%\Packages\OpenAI.Codex_2p2nqsd0c76g0\web\Codex\Default\{Cache\Cache_Data\data_2, Preferences, Local State}
레지스트리	SystemAppData\Helium\User.dat
산출물	%USERPROFILE%\Documents\Codex\<date>\<slug>\{outputs, tmp\pdfs}
실행 흔적	Prefetch\CODEX.EXE-*.pf

● Gemini Web — Chrome 프로파일

저장소	아티팩트 / 추출 정보
History	gemini.google.com/app/<convId> · /u/<n>/app · accounts.google.com/Logout · ServiceLogin · accountchooser · myaccount.google.com/profile/name/edit
Cookies	SynthID · COMPASS 포함
환경설정	Preferences · Secure Preferences
캐시	Cache\Cache_Data → f_* (생성 이미지 원본) · sqldb0
다운로드	downloads_url_chains · Zone.Identifier ADS
파일시스템	\$MFT

● LM Studio — 로컬 LLM 데스크톱 앱

루트 ~\.lmstudio\

저장소	아티팩트 / 추출 정보
대화	conversations*.conversation.json
모델·자격	models · credentials\lmstudio-hub.json
서버 로그	server-logs\<YYYY-MM>\<YYYY-MM-DD>.<n>.log
RAG	.internal\retrieval-database.db · .internal\temp
업로드 파일	user-files\<id>\<name>.metadata.json
확장	extensions · config-presets · skills
계정 프로필	user-profile.json · local-identity.json · lm-link-account-status-cache.json · lm-link-config.json
설치·실행	%APPDATA%\LM Studio\lockfile · %LOCALAPPDATA%\Programs\LM Studio · %LOCALAPPDATA%\lm-studio-updater\installer.exe

PART 6. 탐지 대상 사용자 행위 및 추적

6.1 파서가 판별하는 사용자 행위

분류	판별 행위	적용 범위
계정	로그인 · 로그아웃 · 계정 전환	전체
애플리케이션 설치	설치 · 삭제	전체
대화 관리	대화 생성 · 삭제 · 수정 · 재시도	전체
프롬프트	프롬프트 입력·전송 · 프롬프트 삭제	전체
파일 입출력	이미지 업로드 · 문서 업로드 · 업로드 파일 삭제 · 문서 다운로드	전체
이미지 생성	이미지 생성 요청 · 생성 이미지 다운로드 · 다운로드 파일 삭제	이미지 생성 기능 보유 클라이언트
공유·내보내기	대화 데이터 Export	전체
	공유 링크 생성 · 대화 공유	Claude Web
실행 상태	애플리케이션 실행 · 정상 종료 · 강제 종료	전체
설정 변경	모델 변경 · 계정 이름 변경	전체
	모델 다운로드 내역	LM Studio
프라이버시	브라우저 시크릿 모드 · 방문기록 삭제 · 캐시 삭제	전체
	임시 채팅 사용	임시 채팅 기능 보유 클라이언트

▼ GUI 출력 화면 구성

기존 상용·오픈소스 포렌식 도구의 검증된 화면 문법을 따르되, LLM 아티팩트 분석에 필요한 요소 (증거 수준·복구 신뢰도·시각 출처·마스킹)를 화면에 노출하도록 설계했다.

항목	내용
제품명	AIFP Examiner (AI Forensic Parser — Examiner)
구성	파서 계층 ① → 백엔드 ②(Docker) → 프론트엔드 GUI ③
구현	HTML + CSS + Vanilla JS · 다크/라이트 테마
산출물	<code>index.html</code> · <code>assets/{styles.css, app.js, data.js}</code> · <code>diagrams/*.html</code>

PART 1. 설계

설계 원칙

#	원칙	화면에 적용된 방식
1	좌 → 중 → 우 단방향 탐색	내비게이션에서 좁히고, 목록에서 고르고, 상세에서 판단한다. 역방향 이동이 없다
2	건수를 먼저 보여준다	좌측 모든 항목에 결과 건수 배지. 0건 대상도 숨기지 않고 표시

#	원칙	화면에 적용된 방식
3	결론 옆에 항상 근거	상세 패널 하단이 근거 아티팩트 목록(경로·오프셋·해시)으로 고정
4	불확실성을 숨기지 않는다	증거 수준·복구 신뢰도·시각 출처를 배지로 상시 노출. 시각 신뢰도가 낮으면 경고
5	읽기 전용을 눈에 보이게	앱바  배지 + 상태바 해시 검증 결과 상시 표시
6	민감정보는 기본 가림	토큰·쿠키·계정 UUID는 마스킹이 기본값. 해제는 명시적 토글

PART 2. 화면 골격

2.1 레이아웃 구조

앱바 AIFP 사건명 · 소스 이미지  읽기전용 전역검색 마스킹 테마 탐색기 탭  대시보드  아티팩트  타임라인  대화  복구  보고서		
내비게이션 244px	증거 패널 가변	상세 패널 336px
분석 대상 행위 분류 증거 수준 (건수 배지)	필터 칩 결과 표 / 타임라인 / 대화	행위 · 확신도 식별 정보 복원 원문 근거 아티팩트
상태바  해시 검증 통과 sha256 도구·규칙 버전 타임존 진행률 6/6		

2.2 공통 요소

요소	역할
앱바 사건 칩	사건명 + 소스 이미지 경로를 항상 노출
 읽기 전용 배지	원본 무변경을 시각적으로 보증
전역 검색	UTF-8 / UTF-16LE 동시 검색을 단일 입력으로
마스킹 토글	토큰·쿠키·개인정보 가림 (기본 ON)
타임존 토글	UTC 저장값을 표시 시점에만 변환
상태바 해시·버전	무결성과 재현성 근거를 하단 고정

PART 3. 화면별 명세

3.1 대시보드

대시보드

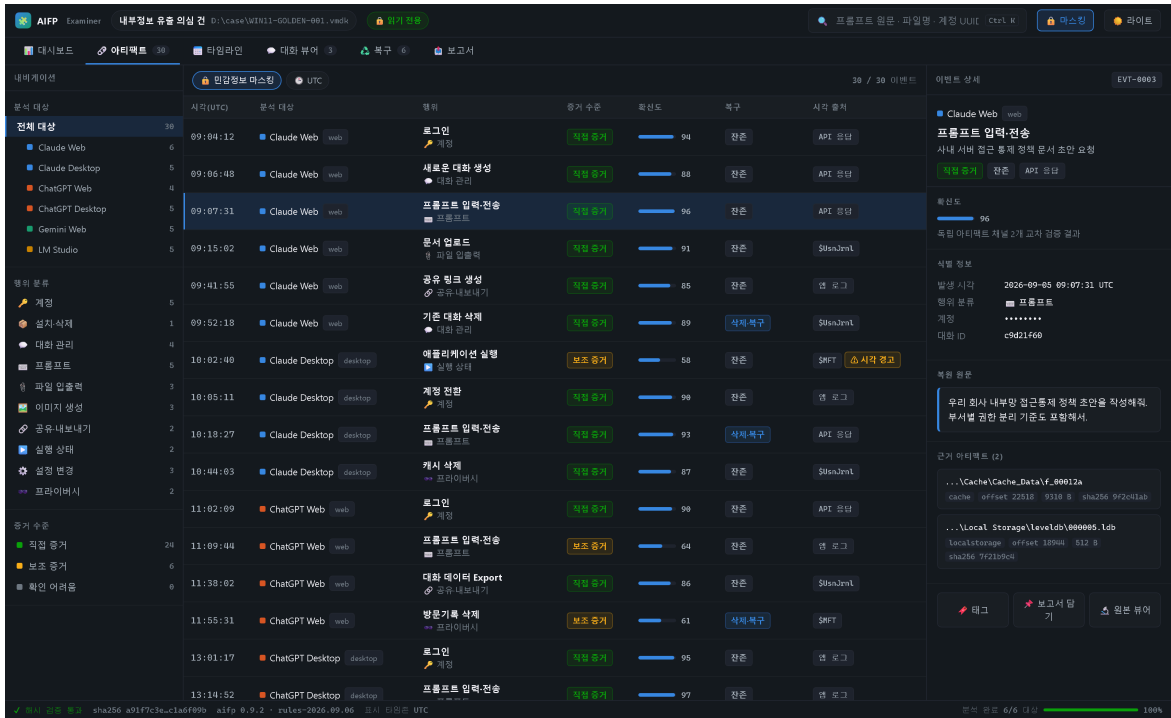


사건 전체를 한 화면에서 판단하는 진입점

구성 요소	내용
KPI 타일 4종	탐지된 분석 대상 · 정규화 이벤트 · 직접 증거 비율 · 삭제 후 복구 건수
서비스별 이벤트	가로 막대. 서비스 색은 화면 전체에서 동일하게 유지되는 정체성
행위 분류별 이벤트	규칙 파일 판정 결과를 분류별로 집계
시간대별 분포	30분 단위 밀도 히스토그램
증거 수준 구성	직접 / 보조 / 확인 어려움 비중을 스택 막대로
검토가 필요한 항목	시각 신뢰도 경고·메타데이터만 복구된 항목을 따로 모아 노출

3.2 아티팩트 탐색기

아티팩트 탐색기



AXIOM의 Navigation → Evidence → Details 3분할을 그대로 따른 주 작업 화면이다.

영역	구성	동작
좌 · 내비게이션	분석 대상(6) / 행위 분류(10) / 증거 수준(3), 각 건수 배치	클릭 시 중앙 목록을 필터링, 재클릭 시 해제
중 · 증거 표	시각 · 분석 대상 · 행위 · 증거 수준 · 확신도 · 복구 · 시각 출처	행 클릭 → 우측 상세. 적용된 필터는 상단 칩으로 표시·개별 해제
우 · 상세	확신도 게이지 → 식별 정보 → 복원 원문 → 근거 아티팩트 → 작업 버튼	근거마다 경로·오프셋·크기·sha256·복구 여부 표기

표 한 행이 담는 정보

열	표기	의미
증거 수준	● 직접 증거 / ● 보조 증거 / ● 확인 어려움	단독으로 행위를 지시하는지 여부
확신도	0~100 막대	독립 아티팩트 채널 수 기반
복구	잔존 / 삭제·복구 / 메타데이터만	원본 잔존 상
시각 출처	\$UsnJrnl \$MFT 앱 로그 API 응답	시각의 출처와 신뢰도
시각 경로	MFT 시각이 앱 종료 시점으로 수렴한 것으로 판단될 때	

3.3 통합 타임라인

통합 타임라인

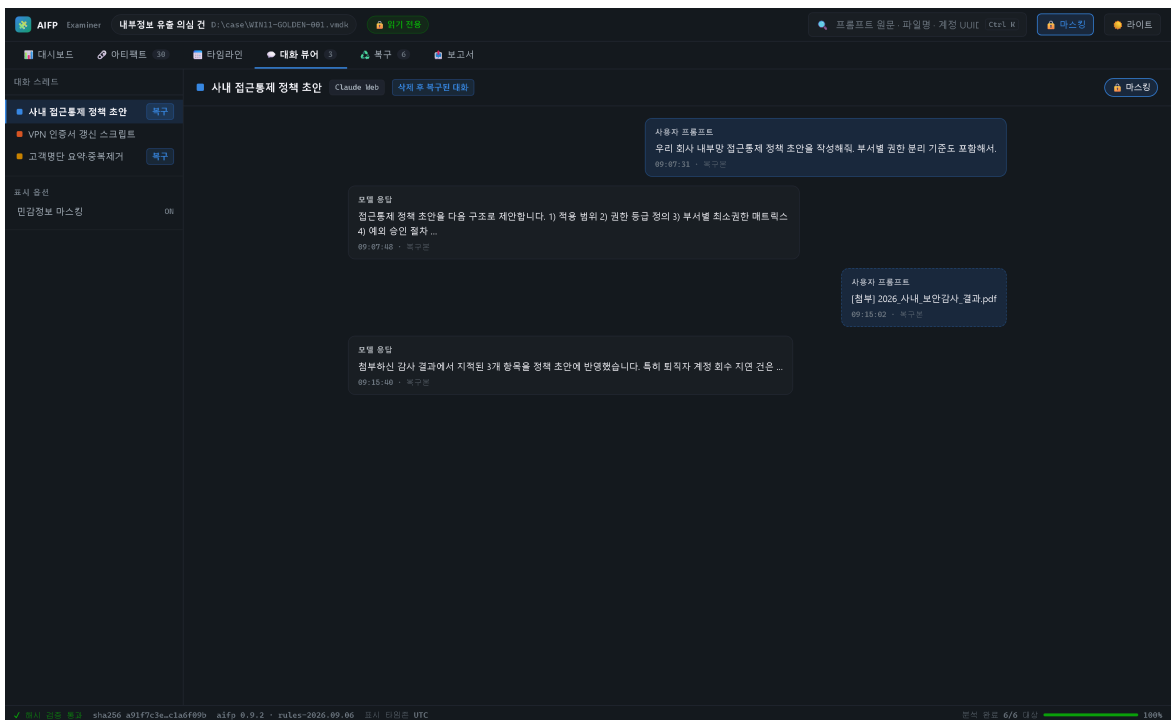


모든 분석 대상의 이벤트를 하나의 시간축에 병합한다.

⚠ 타임라인에서 가장 중요한 건 **시각의 출처**다. 같은 09:52라도 `$UsnJrnl` 기록과 MFT 시각은 신뢰도가 다르다. 모든 항목에 출처 배지를 붙이고, 낮으면 경고를 함께 표시한다.

3.4 대화 뷰어

대화 뷰어

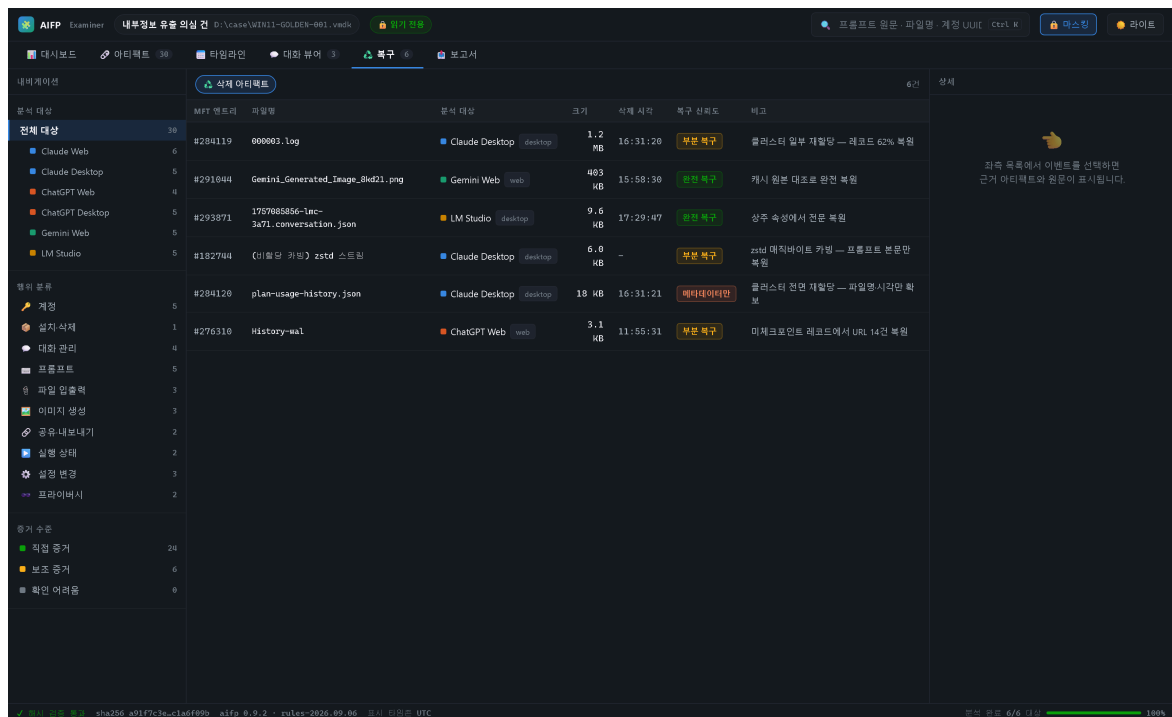


복원된 프롬프트·응답을 실제 대화 형태로 재구성한다. 조사관이 “무엇을 질의했는가”를 읽는 화면이다.

요소	설명
좌측 스레드 목록	서비스 색 점 + 제목 + 복구 여부 배지
말풍선	사용자 프롬프트(우측 강조) / 모델 응답(좌측) 구분
첨부 표시	업로드 파일은 점선 테두리로 구분
잔존 / 복구본 표기	각 메시지 하단에 원본 잔존인지 복원본인지 명시

3.5 복구

복구



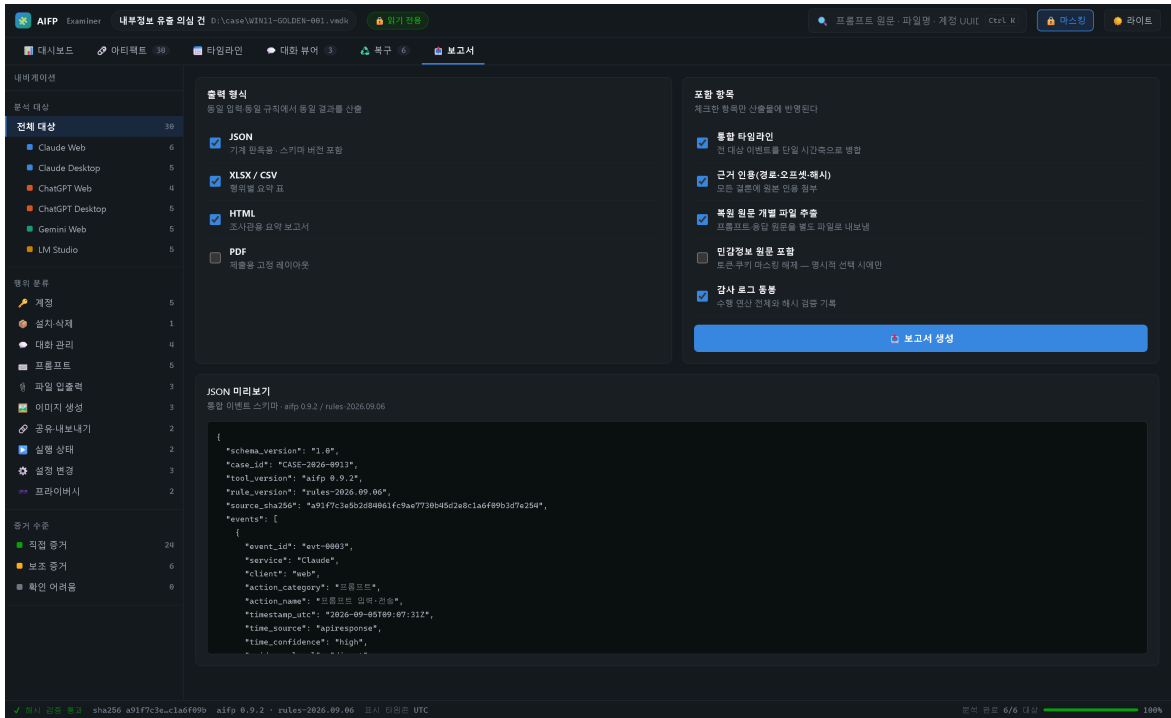
삭제 아티팩트를 복구 신뢰도와 함께 제시한다.

복구 등급	배지	의미
완전 복구	●	원본과 동일한 내용 확보
부분 복구	●	클러스터 일부 재할당 등으로 일부만 확보
메타데이터만	●	파일명·크기·시각만 확보, 본문 소실

비고 열에 왜 그 등급인지를 적는다. “클러스터 일부 재할당 — 레코드 62% 복원”처럼 근거가 함께 있어야 보고서에 인용할 수 있다.

3.6 보고서

보고서



영역	구성
출력 형식	JSON · XLSX/CSV · HTML · PDF 선택
포함 항목	통합 타임라인 · 근거 인용 · 원문 추출 · 감사 로그
민감정보 원문 포함	기본 해제 상태. 체크해야만 마스킹이 풀린다
JSON 미리보기	통합 이벤트 스키마 실제 출력 형태를 즉시 확인

3.7 라이트 테마

라이트 테마

제출용 캡처·인쇄를 고려해 라이트 테마를 별도로 설계했다.

PART 4. 시스템 흐름

4.1 아키텍처

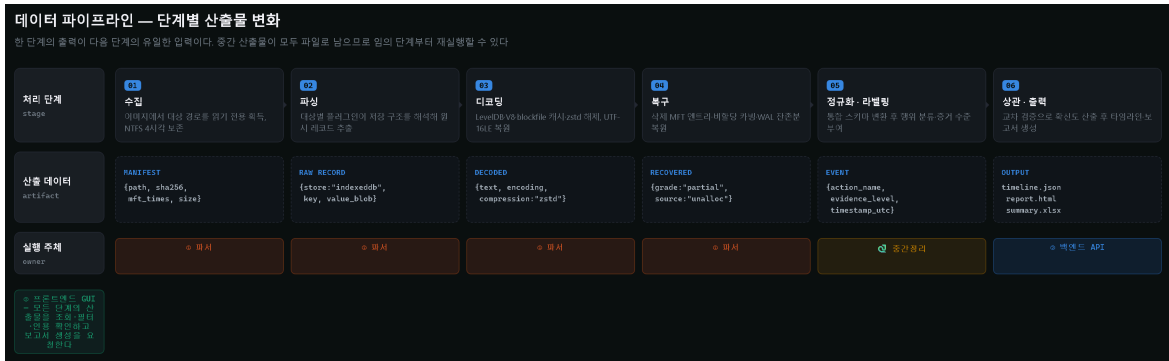
아키텍처

계층	구성	역할
① 파서 계층	대상별 파서 플러그인 · 디코더 · NTFS 계층 · 복구기	이미지에서 원시 레코드(<code>raw_records/*.jsonl</code>)를 뽑는다. 대상별 1 프로세스로 분리해 하나가 죽어도 나머지가 계속된다

계층	구성	역할
중간정리 프로그램	<code>normalizer / labeler</code>	파서 산출물을 통합 이벤트 스키마로 정규화하고, 규칙 파일을 해석해 행위 카테고리이징 · 증거 수준 라벨링 · 확신도 산출을 수행한다. 백엔드의 첫 단계로 컨테이너 안에서 동작
② 백엔드 (Docker)	API 서버(FastAPI) · 작업 큐(Redis+워커) · 이벤트 저장소(PostgreSQL) · 산출물 볼륨	컨테이너 4종. 외부 egress 차단으로 오프라인 동작을 구조적으로 보장
③ 프론트엔드 GUI	본 문서의 6개 화면	REST로 조회, WebSocket으로 진행률 구독

4.2 데이터 파이프라인

데이터 파이프라인

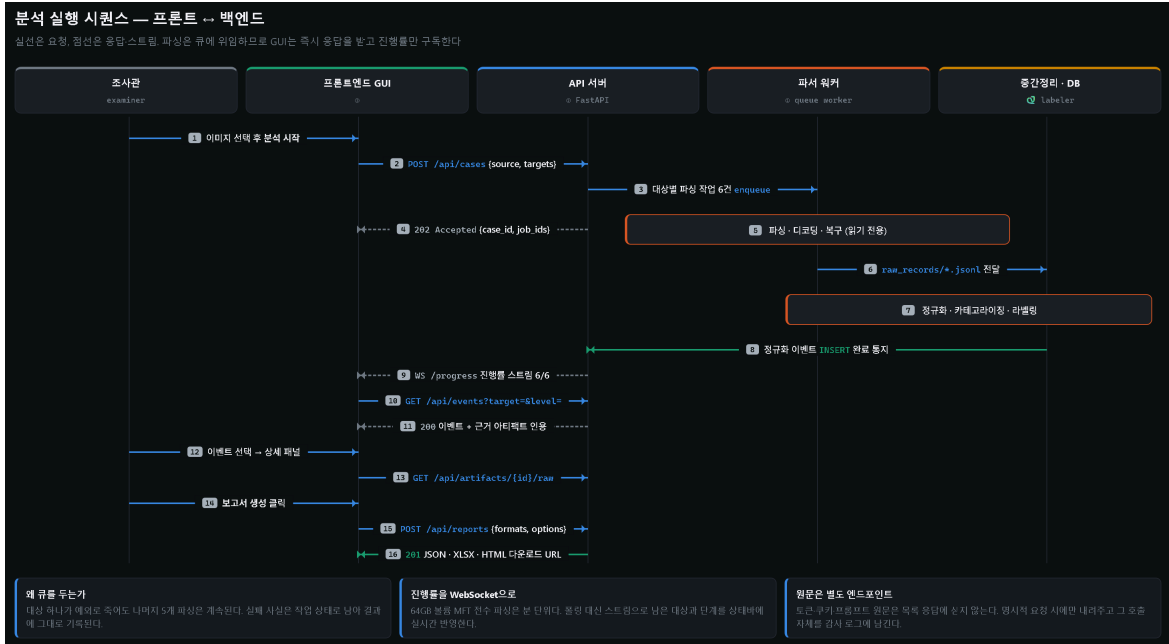


각 단계의 산출물이 모두 파일로 남는다. 따라서 임의 단계부터 재실행이 가능하다. 규칙 파일만 고쳐서 05단계부터 다시 돌리면 파싱을 반복하지 않아도 된다.

단계	실행 주체	산출 데이터
01 수집	① 파서	<code>manifest</code> — 경로·해시·MFT 4시각
02 파싱	① 파서	<code>raw record</code> — 저장소별 원시 키/값
03 디코딩	① 파서	<code>decoded</code> — 평문 + 인코딩·압축 정보
04 복구	① 파서	<code>recovered</code> — 복구 등급 + 출처
05 정규화·라벨링	중간정리	<code>event</code> — 통합 이벤트 스키마
06 상관·출력	② 백엔드	<code>timeline.json</code> · <code>report.html</code> · <code>summary.xlsx</code>

4.3 실행 시퀀스

실행 시퀀스



구간	흐름
분석 시작	GUI → POST /api/cases → API가 대상별 작업 6건을 큐에 등록 → 즉시 202 반환
백그라운드 처리	워커가 파싱·디코딩·복구 → 중간정리가 정규화·라벨링 → DB 적재
진행률	WS /progress 로 남은 대상과 단계를 상태바에 실시간 반영
조회	GET /api/events?target=&level= — 목록 응답에는 원문·토큰을 실지 않는다
원문 열람	GET /api/artifacts/{id}/raw — 명시적 요청 시에만. 이 호출 자체가 감사 로그에 남는다
보고서	POST /api/reports → 다운로드 URL 반환

4.4 API 요약

메서드	엔드포인트	용도
POST	/api/cases	사건 생성 + 분석 작업 등록
GET	/api/cases/{id}	사건 메타·해시·도구/규칙 버전
GET	/api/events	이벤트 목록 (대상·분류·증거수준·기간 필터)
GET	/api/events/{id}	단일 이벤트 + 근거 아티팩트
GET	/api/artifacts/{id}/raw	원문·바이너리 (마스킹 해제 시 감사 로그 기록)
GET	/api/threads/{id}	대화 스레드 재구성 결과
GET	/api/recovered	삭제 아티팩트 복구 목록
POST	/api/reports	보고서 생성 요청
WS	/api/progress/{case_id}	진행률 스트림

PART 5. 구현 및 다음 단계

5.1 파일 구조

AIFP_GUI/	
├ index.html	화면 골격 (앱바 · 탭 · 3분할 · 상태바)
├ assets/	
│ └ styles.css	디자인 토큰 + 레이아웃 + 컴포넌트 (다크/라이트)
│ └ app.js	상태 → 렌더 단방향, 탭/필터/상세/차트
│ └ data.js	목록 데이터 (실제로는 API 응답으로 대체)
├ diagrams/	
│ └ architecture.html	시스템 아키텍처
│ └ pipeline.html	데이터 파이프라인
│ └ sequence.html	실행 시퀀스
└ shots/	렌더링 이미지 (본 보고서 삽입본)

5.2 기술 선택

계층	선택	이유
프론트	Vanilla JS + CSS 변수	외부 CDN 의존이 없어야 오프라인 조사 환경에서 그대로 뜬다. 프레임워크 도입은 상태가 복잡해진 뒤에 판단
차트	직접 구현(DOM/CSS)	막대·히스토그램·스택 3종뿐이라 라이브러리가 과하다.
테마	<code>data-theme</code> 속성 + CSS 변수	한 곳에서 값만 교체
백엔드	FastAPI + PostgreSQL + Redis	파이썬 파서와 같은 런타임, 비동기 스트림(WS) 기본 지원
배포	Docker Compose	라이브러리 버전 고정 = 재현성 확보

참고 자료

- [Autopsy User Documentation — UI Layout](#)
- [Autopsy User Documentation — Tree Viewer](#)
- [Autopsy User Documentation — Result Viewer](#)
- [Magnet AXIOM — Examining evidence](#)
- [Magnet AXIOM — Viewing the timeline](#)
- [Magnet AXIOM Cyber 리뷰 — Forensic Focus](#)